Adaptation-Aware Model Selection for Few-Shot Digital Twin Instantiation

ARTICLE INFO

Keywords:
digital twin
few-shot adaptation
model instantiation
architecture selection
model deployment limits
heterogeneous computing centers

ABSTRACT

As an important modeling tool for managing computing centers, digital twin (DT) platforms reuse pre-trained DT models from existing centers to reduce the cost of DT instantiation in large-scale computing networks. Target-center heterogeneity in resource configurations and workload characteristics complicates direct model reuse when target adaptation uses a small support set. However, existing few-shot adaptation methods typically retain a predefined source architecture and therefore do not directly address target-specific model selection for heterogeneous centers under per-model deployment limits. To address this challenge, we propose Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI), an adaptation-aware model-selection algorithm for DT instantiation with few-shot target adaptation under deployment limits. MSA-DTI constructs a reusable candidate bank of source-trained models with diverse architectures, filters candidates that violate the operation-count and parameter-count limits, and adapts the remaining candidates using the same target support set and update budget. Among candidates feasible across the supported configurations, the one with the lowest mean loss after adaptation on development centers serves as the reference. A target-specific alternative replaces it only if its relative reduction in validation loss reaches a calibrated threshold. On the Bitbrains workload trace, MSA-DTI reduces aggregate mean squared error (MSE) by 6.04% compared with pre-training and fine-tuning (PT-FT), while satisfying both deployment limits in all evaluated cases.

1. Introduction

As an important modeling tool for managing computing centers, a shared digital twin (DT) platform maintains center-specific models for monitoring, prediction, analysis, and decision making (Khan et al., 2022; Mihai et al., 2022; Tao et al., 2019; Xu et al., 2023). In large-scale computing networks, reusing and adapting pre-trained DT models instead of constructing each target model from scratch reduces DT-instantiation cost (Giannetti and Essien, 2022; Sartaj et al., 2025). Each pre-trained model comprises an architecture and source-trained parameters. The architecture determines the model structure and deployment-related complexity, whereas the parameters provide the initialization for target adaptation. For a new or reconfigured center, differences in resource configurations, monitored-variable availability, workload patterns, and data distributions complicate direct reuse of a source model. The source architecture is therefore not necessarily suitable for the target center. The target provides a small support set for adaptation and a separate validation set for comparing adapted candidates. In addition, large-scale DT platforms impose per-model resource budgets, represented here by limits on estimated operation count per inference and parameter count. Target-center instantiation therefore requires selecting a suitable reusable model and adapting its parameters from the small support set while satisfying both deployment limits. Figure 1 illustrates this setting.

Existing few-shot adaptation methods reuse pre-trained models (Gharoun et al., 2024; Vettoruzzo et al., 2024) and have also been applied to DT modeling (Sartaj et al., 2025). However, these methods typically retain the source architecture and therefore do not directly address target-specific model selection for heterogeneous target centers under deployment limits. Deployment-aware architecture-selection methods consider model performance together with deployment cost (Tan et al., 2019; Wu et al., 2019; Zhao et al., 2024), but do not directly address few-shot adaptation of reusable pre-trained models. Methods that jointly consider architecture decisions and few-shot adaptation (Eustratiadis et al., 2024; Lian et al., 2020; Zhao et al., 2022) typically use target-side search to design an architecture or decide which model components to fine-tune or augment (the adaptation structure). The remaining challenge is therefore to select a suitable reusable pre-trained DT model under per-model deployment limits and adapt its parameters from a small target support set.

To address this challenge, we propose Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI), an algorithm that selects reusable models by comparing them after few-shot adaptation under deployment limits. MSA-DTI first constructs a reusable candidate bank of source-trained models with diverse architectures. For a target center, it filters candidates that violate the deployment limits and adapts the remaining feasible candidates under a common

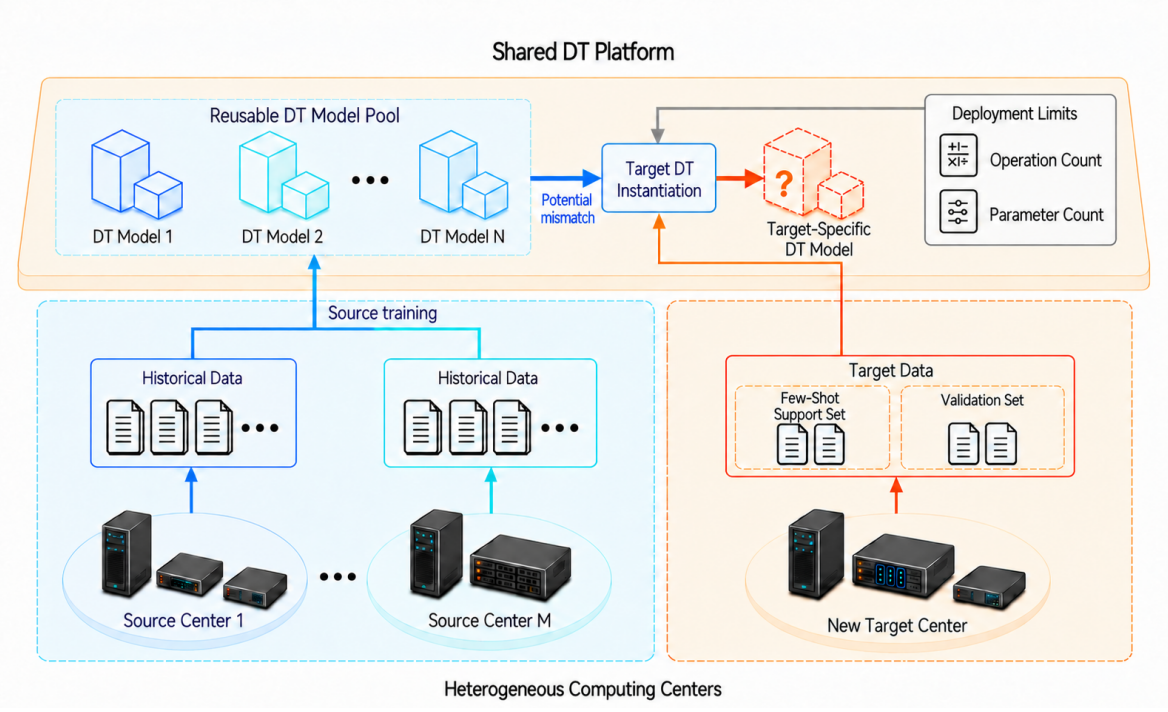


Figure 1: System setting for target-specific DT instantiation.

adaptation budget. The reference is the candidate the platform would use if every center adapted the same source model. Development centers determine which candidate serves this role. Another candidate can be better suited to a particular target, but its validation advantage must meet a calibrated relative margin before it replaces the adapted reference.

The main contributions are summarized below.

- We formulate DT instantiation as jointly selecting among reusable models with different architectures, all trained on source data, and adapting the selected model's parameters from a small target support set, subject to hard limits on the operation and parameter counts of each model.
- We propose a model selection strategy that first filters out infeasible candidates and then compares the remaining candidates after adapting them using the same target support set and update budget.
- We formulate target-specific replacement relative to a common, development-selected candidate. A multiplicative perturbation model for post-adaptation losses yields a relative improvement threshold, which we calibrate on separate centers.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 defines the system model and problem. Section 4 presents MSA-DTI. Section 5 reports the experiments, and Section 6 concludes the paper.

2. Related Work

2.1. Limited-Data Model Adaptation

Limited-data methods leverage source or prior knowledge for target-model adaptation (Gharoun et al., 2024; Son et al., 2025; Vettoruzzo et al., 2024). Recent approaches improve cross-task and cross-domain adaptation through context modeling and transferable representations (Wang et al., 2023; J. Wu et al., 2024; Zhao et al., 2023; Zintgraf et al., 2019). Limited-data learning has also been applied to DT construction and workload prediction (Sartaj et al., 2025; Zuo et al., 2025). Most parameter-adaptation methods operate with a fixed architecture. For heterogeneous target centers, a fixed architecture is not necessarily suitable, motivating selection among reusable source-trained models.

Pre-trained model selection has also been studied in model hubs: LogME ranks candidates by estimated transferability before fine-tuning (You et al., 2021), while B-Tuning uses multiple highly ranked pre-trained models to guide target-model tuning (You et al., 2022).

A candidate's loss on a finite validation set estimates its performance on unseen data. Estimation error can change the ordering of candidates, particularly when their losses are close (Bornschein et al., 2020; Lacoste et al., 2014). This motivates a conservative decision rule when adapted reusable models are compared on target validation data.

2.2. Digital Twin Deployment and Architecture Selection

In communication and computing systems, digital twins represent system states and services while supporting model updates (Hakiri et al., 2024; J. Li et al., 2024; Qin et al., 2024; Wu et al., 2021). At the deployment level, related work considers service orchestration and computation offloading under resource constraints (Emami Khansari and Sharifian, 2025; Hortelano et al., 2023). Together, these strands motivate jointly considering model suitability and deployment feasibility in target-specific DT instantiation.

Deployment-aware neural architecture search (NAS) evaluates model performance together with deployment cost (Tan et al., 2019; Wu et al., 2019; Zhao et al., 2024). Few-shot, zero-shot, and training-free NAS variants reduce the cost of architecture evaluation during search (G. Li et al., 2024; M.-T. Wu et al., 2024; Zhao et al., 2021). In Zhao et al. (2021), each of a few networks evaluates a subset of candidate architectures. In this paper, "few-shot" refers to the small labeled support set used for target adaptation.

Among meta-learning-based NAS methods, T-NAS learns transferable architecture knowledge for new-task adaptation (Lian et al., 2020). H-Meta-NAS further incorporates hardware constraints into task-adaptive search (Zhao et al., 2022). Neural fine-tuning search instead starts from a pre-trained backbone and searches over adaptation structures (Eustratiadis et al., 2024). Together, these methods extend task adaptation beyond parameter updates to architecture or adaptation-structure decisions.

To address target-specific model selection under deployment limits without target-stage search for new architectures or adaptation structures, MSA-DTI selects among a finite bank of separately source-trained reusable models using deployment feasibility and post-adaptation performance.

3. System Model and Problem Formulation

3.1. System Setting and Target Model

We consider a shared DT platform that maintains center-specific models for heterogeneous computing centers. Let c denote a new or reconfigured target center. To enable model reuse across centers with different monitored-variable availability, all observations follow a common metric schema. A binary mask records which variables are available, so missing variables do not change the input structure. Let $\mathcal{X}$ denote the common input space defined by this metric schema, and let $\mathcal{Y}$ denote the output space of the DT task. Let $\mathcal{A}$ denote the set of model architectures available to the shared DT platform. For $a \in \mathcal{A}$, let $\Theta(a)$ denote the trainable-parameter space associated with architecture a, and let $\theta \in \Theta(a)$ denote a particular parameter setting. A model is represented as

$$f_{a,\theta} : \mathcal{X} \to \mathcal{Y}, \qquad a \in \mathcal{A},\ \theta \in \Theta(a). \tag{1}$$

The DT model instantiated for center c is represented as

$$\mathrm{DT}_c = \left(a_c, \theta_c\right), \qquad a_c \in \mathcal{A},\ \theta_c \in \Theta(a_c), \tag{2}$$

where a_c and θ_c denote the target architecture and target-specific parameters, respectively. During instantiation, the available target data are represented by a support set and a validation set.

$$\begin{aligned} \mathcal{D}_c^{\mathrm{sup}} &= \left\{ (x_{c,i}^{\mathrm{sup}}, y_{c,i}^{\mathrm{sup}}) \right\}_{i=1}^{K_c}, \\ \mathcal{D}_c^{\mathrm{val}} &= \left\{ (x_{c,j}^{\mathrm{val}}, y_{c,j}^{\mathrm{val}}) \right\}_{j=1}^{V_c}, \\ \mathcal{D}_c^{\mathrm{tar}} &= \mathcal{D}_c^{\mathrm{sup}} \cup \mathcal{D}_c^{\mathrm{val}}, \qquad \mathcal{D}_c^{\mathrm{sup}} \cap \mathcal{D}_c^{\mathrm{val}} = \varnothing. \end{aligned} \tag{3}$$

In these sets, $(x_{c,i}^{\mathrm{sup}}, y_{c,i}^{\mathrm{sup}})$ and $(x_{c,j}^{\mathrm{val}}, y_{c,j}^{\mathrm{val}})$ denote the ith support and jth validation input-output pairs, respectively. Their union $\mathcal{D}_c^{\mathrm{tar}}$ denotes the target data available during instantiation. Here, $K_c, V_c \in \mathbb{N}_+$ denote the support and

validation sample counts, respectively. Support samples are used for few-shot parameter adaptation, whereas validation samples are reserved for post-adaptation candidate comparison. For a forecasting task, each validation example comprises an input window and its subsequent target values. It becomes available for selection only after all of those target values have been observed. Future evaluation labels are excluded from $\mathcal{D}_c^{\mathrm{tar}}$. The present study concerns determining (a_c, θ_c) during target-center instantiation. Subsequent model updates are outside its scope.

3.2. Reusable Models and Deployment Limits

The shared DT platform maintains source-trained models learned from observations of existing centers. Each reusable model consists of a model architecture and its source-trained parameter initialization and serves as a candidate for target-center instantiation. Let the finite set of N reusable source-trained models be

$$\mathcal{Q} = \{q_n\}_{n=1}^{N}, \qquad q_n = (a_n, \theta_n^0), \tag{4}$$

where q_n denotes the nth reusable source-trained model, $a_n \in \mathcal{A}$ denotes its architecture, and $\theta_n^0 \in \Theta(a_n)$ denotes its source-trained initialization. The architectures represented in $\mathcal{Q}$ form the set $\mathcal{A}_{\mathcal{Q}} = \{a_n\}_{n=1}^{N} \subseteq \mathcal{A}$.

For target center c, the platform specifies two per-model deployment limits: a maximum estimated operation count for a forward pass on one input example, and a maximum number of trainable parameters. Let B_c^F and B_c^P denote these limits, respectively. For $a \in \mathcal{A}_{\mathcal{Q}}$, let $F_c(a)$ denote the estimated operation count, and let $P_c(a)$ denote the exact number of trainable parameters. In this formulation, parameter adaptation only changes parameter values while keeping the architecture fixed. Therefore, $F_c(a)$ and $P_c(a)$ remain unchanged.

For center c, the deployment-feasible reusable model set is

$$\mathcal{Q}_c^{\mathrm{fea}} = \left\{ q_n \in \mathcal{Q} \;\middle|\; F_c(a_n) \le B_c^F,\ P_c(a_n) \le B_c^P \right\}. \tag{5}$$

Both deployment limits are modeled as hard constraints rather than penalty terms.

3.3. DT Model Instantiation Problem

Let $\mathbb{P}_c$ denote the target-center data distribution over $\mathcal{X} \times \mathcal{Y}$, and let $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\ge 0}$ denote the task loss function. The target risk is defined as

$$R_c(a, \theta) = \mathbb{E}_{(x,y)\sim\mathbb{P}_c}\left[\ell(f_{a,\theta}(x), y)\right]. \tag{6}$$

Let $\mathcal{U}_c$ denote a prescribed target-adaptation operator. For each reusable model $q_n = (a_n, \theta_n^0)$, $\mathcal{U}_c(q_n, \mathcal{D}_c^{\mathrm{sup}}) \in \Theta(a_n)$ denotes the target-specific parameters obtained by adapting q_n using the target support set. When $\mathcal{Q}_c^{\mathrm{fea}} \neq \emptyset$, an ideal reusable-model decision for center c is defined as

$$\begin{aligned} n_c^{\dagger} \in \underset{n\in\{1,\dots,N\}}{\arg\min} \quad & R_c\left(a_n, \mathcal{U}_c(q_n, \mathcal{D}_c^{\mathrm{sup}})\right) \\ \text{s.t.} \quad & F_c(a_n) \le B_c^F, \\ & P_c(a_n) \le B_c^P. \end{aligned} \tag{7}$$

This ideal decision requires the unknown target distribution $\mathbb{P}_c$. A common policy $\pi_q(c) = q$ fixes the same candidate identity for every center but adapts its parameters on each center's own support set. Let $\mathcal{Q}_{\mathrm{all}}$ be the nonempty set of candidates feasible in every task and budget configuration supported by the policy. Let $\bar{L}_c(q)$ be the post-adaptation risk averaged over the tasks at center c. The best common policy minimizes $\mathbb{E}_C[\bar{L}_C(q)]$ over $q \in \mathcal{Q}_{\mathrm{all}}$, where C denotes a target center drawn from the population of interest. Section 4.2 uses development centers to estimate this policy and fixes its candidate q^{ref}. Another candidate can still perform better at an individual target.

When the reference is feasible and replacement is enabled, we assess a switch by its excess loss over the common policy. To account for uncertainty in the target ranking, let $\mathcal{S}_c$ be a set of plausible loss vectors for the adapted feasible candidates, with $s(q)$ the loss assigned to q. We compare actions by their worst-case excess over the common policy, a baseline-relative criterion also studied in sequential decision making (Ghavamzadeh et al., 2016):

$$W_c(q) = \sup_{s\in\mathcal{S}_c}\{s(q) - s(q^{\mathrm{ref}})\}, \qquad q_c^* \in \arg\min_{q\in\mathcal{Q}_c^{\mathrm{fea}}} W_c(q). \tag{8}$$

Retaining the reference has zero excess for every s. A replacement can therefore be chosen only if its worst-case excess is nonpositive. Section 4.3 specifies $\mathcal{S}_c$ from the validation losses and derives the corresponding selection rule.

4. The Proposed MSA-DTI Algorithm

4.1. Overview

Figure 2 summarizes MSA-DTI for the DT model instantiation problem. The first module builds a bank of source-trained models and designates a deployment reference using development data. The second removes candidates that exceed either target deployment limit. The third adapts the remaining models on the same support set. It replaces the adapted reference only when another model reduces validation loss by a calibrated proportion.

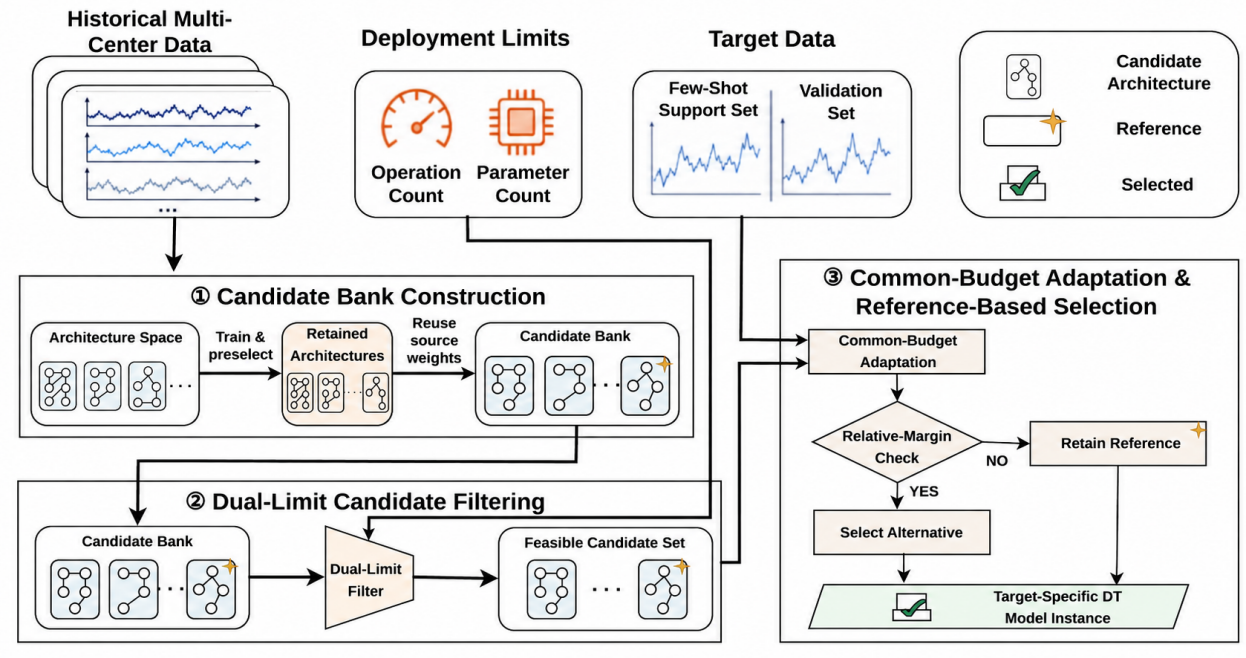


Figure 2: Overall workflow of MSA-DTI.

4.2. Candidate Bank Construction

MSA-DTI first trains each architecture on the source centers. On a separate group of centers, it adapts each model to each task and measures validation MSE. It averages the four task losses within each center and then weights centers equally. The best model that fits the supported budgets serves as the screening reference a^{pre}. A separate group supplies the screening cases. Other centers are reserved for designation of the deployment reference, margin calibration, and evaluation. The screening reference sets a common standard for retaining a challenger. For each screening case, all feasible models start from their source checkpoints and adapt on the support set. The reference must fit the case budget. A separate validation set ranks the adapted models. A later check set tests whether an apparent gain over the reference persists.

Each challenger can earn evidence in two ways. It can have the lowest check loss among feasible models. It can also win on validation with lower loss than the reference. In either case its check loss must be lower than the reference's. The bank retains the reference and any challenger that reaches the preset count for either type of evidence.

Each retained architecture a_q keeps the parameters θ_q^0 learned on the source centers. The pair $q = (a_q, \theta_q^0)$ is a reusable candidate. When the output head depends on the forecast horizon, the candidate identifier selects the corresponding checkpoint. In Figure 3, the blue outlines link the retained architecture a^{ref} to its candidate record q^{ref}, later chosen as the deployment reference.

We estimate the common policy from Section 3.3 within the jointly feasible set $\mathcal{Q}_{\mathrm{all}}$. Let $\mathcal{V}_{\mathrm{dev}}$ denote the nonempty set of development target instances and $\mathcal{H}_v$ the nonempty set of prespecified tasks associated with instance v. For candidate q, $Y_{v,h}^{\mathrm{real}}(q)$ denotes its post-adaptation loss measured on the development assessment observations for task h,

withheld from that task's adaptation and validation comparison. Its average development loss is

$$\overline{Y}_{\mathrm{dev}}(q) = \frac{1}{|\mathcal{V}_{\mathrm{dev}}|} \sum_{v \in \mathcal{V}_{\mathrm{dev}}} \frac{1}{|\mathcal{H}_v|} \sum_{h \in \mathcal{H}_v} Y^{\mathrm{real}}_{v,h}(q), \qquad q \in \mathcal{Q}_{\mathrm{all}}. \tag{9}$$

Reference designation requires $\mathcal{Q}_{\mathrm{all}} \neq \varnothing$ and uses

$$q^{\mathrm{ref}} \in \arg\min_{q \in \mathcal{Q}_{\mathrm{all}}} \overline{Y}_{\mathrm{dev}}(q). \tag{10}$$

If candidates have exactly the same average development loss, we first compare their maximum parameter counts across task configurations, preferring the smaller value. If a tie remains, we compare their maximum operation counts across the same configurations, again preferring the smaller value. Any remaining tie is resolved by the lower architecture index. Let r index the selected reference record, written as $q^{\mathrm{ref}} = (a^{\mathrm{ref}}, \theta^0_r)$ for the corresponding task. This minimizes empirical assessment loss among policies that use one candidate identity across the specified configurations. The identity is fixed before calibration and evaluation. Its parameters are adapted at each target, as for every other feasible candidate. This stage outputs $\mathcal{Q}$ and q^{ref} for target-side instantiation.

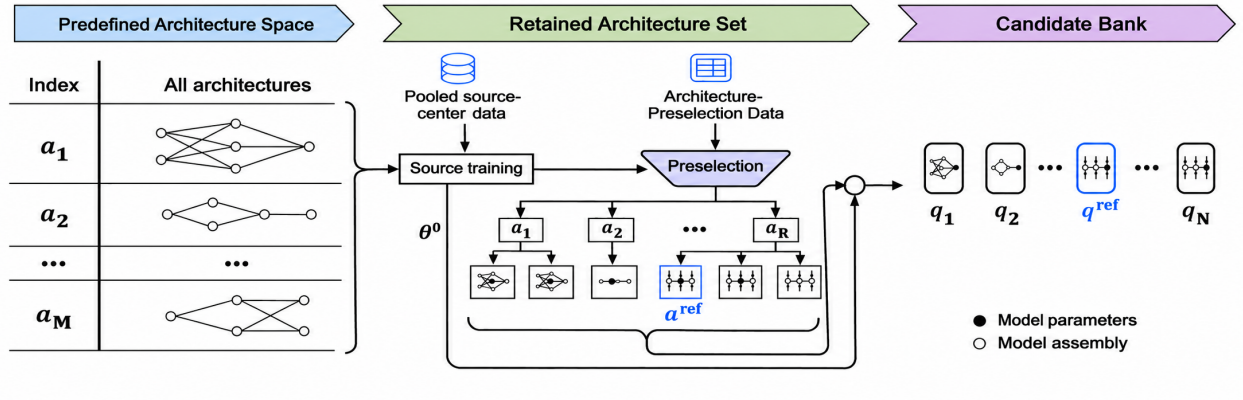


Figure 3: Candidate bank and reference selection.

4.3. Target-Side Instantiation

For target center c, MSA-DTI computes $\mathcal{Q}^{\mathrm{fea}}_c$ using Eq. (5) and restricts target adaptation to the candidates in this set. Because the subsequent reference-based selection requires a deployment-feasible reference for comparison and fallback, MSA-DTI proceeds only if $q^{\mathrm{ref}} \in \mathcal{Q}^{\mathrm{fea}}_c$. Otherwise, it terminates without DT instantiation under the current deployment limits.

For target-center instantiation, the support set $\mathcal{D}^{\mathrm{sup}}_c$ is used for parameter adaptation, whereas the validation set $\mathcal{D}^{\mathrm{val}}_c$ is used for post-adaptation candidate comparison. For a dataset $\mathcal{D}$, let

$$\mathcal{L}(\mathcal{D}; a, \theta) = \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \ell\left(f_{a,\theta}(x), y\right) \tag{11}$$

denote the average task loss over the examples in $\mathcal{D}$.

For each feasible candidate $q = (a_q, \theta^0_q)$, let $\theta^{(t)}_{c,q}$ denote its parameters after t target-side adaptation steps. With a common update budget of T steps and a learning rate η shared by all feasible candidates, the adaptation is

$$\begin{aligned} \theta^{(0)}_{c,q} &= \theta^0_q, \\ \theta^{(t+1)}_{c,q} &= \theta^{(t)}_{c,q} - \eta \nabla_\theta \mathcal{L}\left(\mathcal{D}^{\mathrm{sup}}_c; a_q, \theta^{(t)}_{c,q}\right), \\ & \qquad t = 0, \ldots, T-1. \end{aligned} \tag{12}$$

Equation (12) gives the gradient-based realization of the target-adaptation operator $\mathcal{U}_c$ introduced in Section 3.3, with $\theta^{(T)}_{c,q}$ as its output. The implementation clips the gradient norm as specified in the experimental setup. After T adaptation steps, let $J_c(q)$ denote the post-adaptation validation score of candidate q.

$$J_c(q) = \mathcal{L}\left(\mathcal{D}^{\mathrm{val}}_c; a_q, \theta^{(T)}_{c,q}\right). \tag{13}$$

To quantify the stability of a validation advantage, consider finite, positive $J_c(q)$ and a relative perturbation radius $0 \le \epsilon < 1$. In Eq. (8), take S_c to be the product of the intervals $[(1-\epsilon)J_c(q), (1+\epsilon)J_c(q)]$ over feasible candidates. Using relative perturbations makes the decision invariant to a common change of loss units.

Replacement criterion. Under this model, a challenger q has nonpositive worst-case excess if and only if $J_c(q) \le (1-\tau)J_c(q^{\mathrm{ref}})$, where $\tau = 2\epsilon/(1+\epsilon)$. To see this, the product set allows the challenger's upper endpoint and the reference's lower endpoint to occur together. Thus, for $q \ne q^{\mathrm{ref}}$,

$$W_c(q) = (1+\epsilon)J_c(q) - (1-\epsilon)J_c(q^{\mathrm{ref}}), \qquad W_c(q^{\mathrm{ref}}) = 0.$$

Requiring $W_c(q) \le 0$ gives the stated threshold. Among challengers, $W_c(q)$ increases strictly with $J_c(q)$. Hence the rule below selects a minimizer of worst-case excess. If the best challenger meets the threshold exactly, it ties with the reference in worst-case excess.

Section 4.4 calibrates $\tau \in [0, 1)$ on held-out centers. The admissible set is

$$\mathcal{A}_c(\tau) = \{q^{\mathrm{ref}}\} \cup \{q \in \mathcal{Q}_c^{\mathrm{fea}} \setminus \{q^{\mathrm{ref}}\} : J_c(q) \le (1-\tau)J_c(q^{\mathrm{ref}})\}. \tag{14}$$

The selected candidate q_c^* has the lowest $J_c(q)$ in this set. For equal validation losses, prefer the smaller parameter count, then the smaller operation count, then the lower architecture index, and finally the lexicographically first candidate identifier. If calibration prescribes retaining the reference in every case (the reference-only policy), set $q_c^* = q^{\mathrm{ref}}$. All feasible candidates must have finite, strictly positive validation scores. If any score is nonfinite or nonpositive, selection terminates with a numerical-domain failure.

The selected candidate q_c^* provides the architecture $a_{q_c^*}$, and its adapted parameters $\theta_{c,q_c^*}^{(T)}$ define the target-specific DT model in Eq. (2). This stage outputs DT_c. Figure 4 illustrates common-budget adaptation and the reference-based selection rule. Candidate labels identify the same models before and after parameter adaptation; the candidate identities and loss values shown are illustrative.

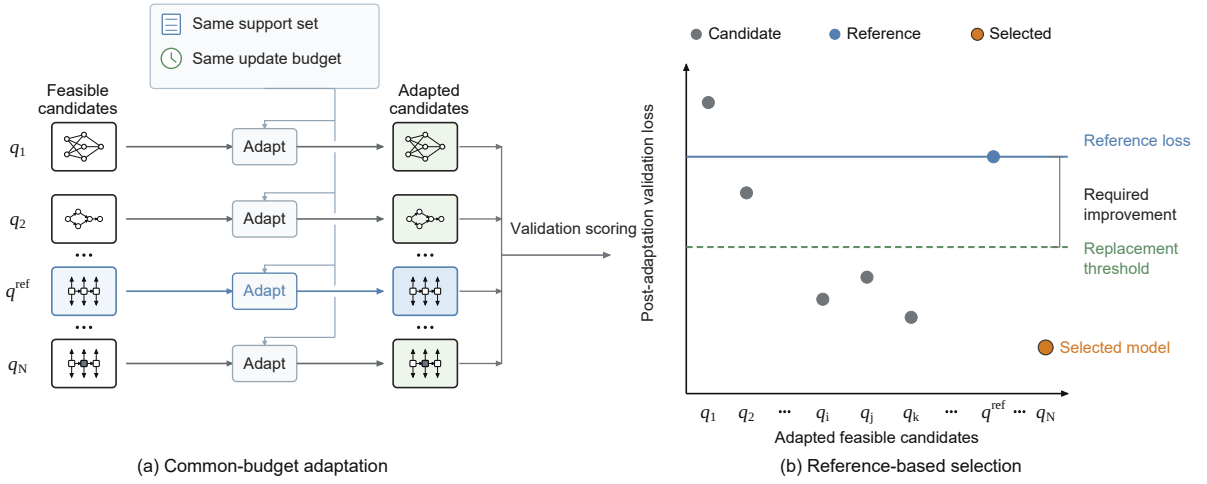


Figure 4: Common-budget adaptation and reference-based selection.

Algorithm 1 summarizes the target-side procedure.

Algorithm 1 Target-Side MSA-DTI for One Center

Require: Candidate bank $\mathcal{Q}$, reference candidate q^{ref}, support set $\mathcal{D}_c^{\mathrm{sup}}$, validation set $\mathcal{D}_c^{\mathrm{val}}$, deployment limits B_c^F and B_c^P, update budget T, learning rate η, and a frozen policy, including its calibrated relative margin τ when the policy is not reference-only

Ensure: Target-specific DT model DT_c, or termination without instantiation

Compute $\mathcal{Q}_c^{\mathrm{fea}}$ using Eq. (5)
if $q^{\mathrm{ref}} \notin \mathcal{Q}_c^{\mathrm{fea}}$ **then**
    terminate without DT instantiation
end if
for all $q \in \mathcal{Q}_c^{\mathrm{fea}}$ **do**
    Adapt q using Eq. (12)
    Compute $J_c(q)$ using Eq. (13)
    if $J_c(q)$ is nonfinite or $J_c(q) \le 0$ **then**
        terminate with a numerical-domain failure
    end if
end for
if the frozen policy is reference-only **then**
    $q_c^* \leftarrow q^{\mathrm{ref}}$
else
    Form $\mathcal{A}_c(\tau)$ using Eq. (14)
    Select $q_c^* \in \arg\min_{q \in \mathcal{A}_c(\tau)} J_c(q)$, using the deterministic tie-breaking order specified above
end if
$\mathrm{DT}_c \leftarrow \left(a_{q_c^*}, \theta_{c,q_c^*}^{(T)}\right)$
return DT_c

4.4. Relative-Margin Calibration and Analytical Properties

The margin τ is calibrated on a pool disjoint from the data used for architecture preselection, reference designation, and final evaluation. Each calibration case uses support data for adaptation and validation examples whose target values are fully observed for model selection. A later segment supplies the outcomes used to compare each selected candidate with the adapted reference during calibration.

Let $\mathcal{T} \subset [0, 1)$ be a preset finite margin set. Let $\widehat{R}_{\mathrm{harm}}(\tau)$ denote the empirical harmful-selection rate and $\widehat{L}(\tau)$ the mean outcome loss of the selected models. Both quantities are averaged first over tasks within each calibration target instance and then equally across instances. A case contributes one to the harm indicator when the selected alternative has a higher outcome loss than the adapted reference, and zero otherwise. Let $\widehat{L}_{\mathrm{ref}}$ denote the corresponding mean outcome loss when the adapted reference is retained in every case. For a preset empirical harm limit ρ, the eligible set is

$$\mathcal{T}_{\mathrm{elig}} = \{\tau \in \mathcal{T} : \widehat{R}_{\mathrm{harm}}(\tau) \le \rho\}. \tag{15}$$

Among eligible margins, select a minimizer of $\widehat{L}(\tau)$, breaking exact ties toward the larger margin. If $\mathcal{T}_{\mathrm{elig}}$ is empty or its minimum loss is not strictly lower than $\widehat{L}_{\mathrm{ref}}$, the policy is reference-only. Otherwise the selected margin is fixed before evaluation. The harm constraint is evaluated empirically on the calibration pool.

For a fixed reference and feasible set, increasing τ can only change a selection from the validation winner to the reference. The fraction of all tasks with a harmful replacement therefore cannot increase, but some beneficial replacements can also be lost. The calibration criterion balances these effects using later observations. It establishes neither coverage of future losses by S_c nor a bound on the future harmful-selection rate. Each instantiated model still satisfies both deployment limits by construction.

With resource counts precomputed for the relevant task configurations, the target-stage computational complexity is

$$O\left(|\mathcal{Q}| + \sum_{q \in \mathcal{Q}_c^{\mathrm{fea}}} \left(T C_q^{\mathrm{grad}} + C_q^{\mathrm{val}}\right)\right). \tag{16}$$

Here, C_q^{grad} and C_q^{val} denote the costs of one target update and validation evaluation for candidate q, respectively. The $|\mathcal{Q}|$ term accounts for deployment filtering.

5. Experimental Evaluation

The experiments assess prediction accuracy, selection reliability, deployment feasibility, and target-side computation. The primary comparison uses the medium deployment budget; tight and loose budgets examine resource sensitivity.

5.1. Experimental Setup

The main experiments use the Bitbrains fastStorage workload trace[1] from the Grid Workloads Archive (Shen et al., 2015). The source pool contains 20 VMs. Four other groups of 20 VMs each supply the screening reference, screening cases, deployment reference, and calibration data, respectively. The 30 evaluation VMs form a separate group. We had already examined their traces in earlier experiments, so this comparison is retrospective.

The trace has a nominal sampling interval of five minutes. Each model uses 96 consecutive records to predict CPU utilization over the next $H \in \{1, 4\}$ records. Parameter adaptation uses $K \in \{10, 20\}$ labeled support windows. A subsequent validation history provides 1342–1351 complete windows before validity masking, all with fully observed prediction horizons at the selection time. The four H/K settings produce 80 calibration tasks and 120 evaluation tasks. Each selected model then makes 621–627 rolling-origin forecasts with fixed parameters. Newly arriving observations enter forecast inputs. The few-shot budget counts only the labels used for parameter updates. Model selection also uses the separate validation history. Future evaluation labels enter neither step.

The target is recorded CPU utilization divided by 100. Inputs comprise ten monitored metrics, two causal CPU features, twelve validity indicators, and a timestamp-gap channel. Invalid measurements are represented by a validity mask and a forward-filled value, initialized at zero. The nine raw metrics other than CPU utilization in percent use source-fitted log scaling. Every method is scored on the same windows with complete, valid target horizons. Table 1 summarizes the main experimental settings.

Table 1: Experimental configuration.

Item	Setting
VM groups	Source / A / B / C / D / E, 20 / 20 / 20 / 20 / 20 / 30
Target input and output	96 records, 25 channels; CPU utilization over $H \in \{1, 4\}$ records
Adaptation support	$K \in \{10, 20\}$ windows; 50 SGD updates, learning rate 0.01
Candidate bank	19 records from 66 source-trained architectures
Reference and margin	Screening reference A55, deployment reference A57, margin $\tau = 0.20$
Budgets	Tight / medium / loose: operations $1.5/5/20 \times 10^6$; parameters $3/10/50 \times 10^4$
Primary evaluation	Medium budget; 120 tasks clustered within 30 VMs

Candidate bank and calibration. All 66 architectures were trained on 20 Bitbrains source VMs for 50 epochs using Adam and MSE. A55 had the lowest mean validation MSE in the first development group. We screened the architectures against A55 on 80 tasks using PyTorch 2.5.1 (Paszke et al., 2019; PyTorch contributors, 2024). We retained the source checkpoints for A55 and the 18 challengers that passed screening. Of these, 10 met both limits at every supported horizon and budget. A57 had the lowest mean assessment MSE among these 10 and became the deployment reference.

Target adaptation used 50 full-support SGD updates with MSE, a learning rate of 0.01 and a gradient norm limit of one. Calibration used 80 tasks at the medium budget. Margins of 15% and 20% achieved the same lowest mean MSE among those meeting the 5% harm criterion. The prespecified tie rule selected 20%, which we used at all three budgets.

The Python scripts use Matplotlib 3.10.0 (Hunter, 2007; The Matplotlib Development Team, 2024) to plot the recorded task results and architecture profiles. The operation estimate counts two operations per multiply–add in linear and convolutional transformations, excluding bias additions and other elementwise operations. The limits in Table 1 apply per model during inference.

Comparison methods. Seven task-matched baselines cover fixed-model adaptation and target-side architecture selection (Table 2). MeDeT-style adaptation uses source meta-training motivated by Sartaj et al. (2025); the H-Meta-NAS-based baseline follows hardware-constrained task-adaptive search (Zhao et al., 2022). Meta+NAS-lite combines a ranking from validation losses on source tasks with one from target support data. The latter uses the entropy of normalized absolute loss gradients as a proxy for candidate quality. The zero-shot baselines use this same support-based

[1] https://atlarge-research.com/gwa-t-12/

proxy without first adapting the models used to compute it. All methods use the same task inputs, available target labels, and resource counts. The fixed-model baselines keep their development-selected backbones across budgets. The search baselines select among architectures that fit the budget for the task.

The SGD baselines used the same target update settings as MSA-DTI. The Adam baselines used a learning rate of 3×10^{-4} with a Huber threshold of 0.2 and a gradient norm limit of 5. After adaptation, search methods ranked candidates by validation MSE, breaking ties by resource counts and then by architecture index.

Table 2: Task-matched baseline implementations.

Method	Initialization and selection	Target updates
PT-FT	Fixed A39, selected on development VMs after source training	SGD/MSE, 50
MeDeT-style	Fixed A57 meta-initialization, selected on development VMs	SGD/MSE, 50
Scratch50	Fixed A54, selected on development VMs and initialized at random	SGD/MSE, 50
Meta+NAS-lite	Up to 12 feasible candidates from the 66 configurations. Rank $0.6\, r_{\text{source}} + 0.4\, r_{\text{support}}$	Adam/Huber, 50 per candidate
Zero-shot NAS	Highest support proxy among feasible candidates, with C1 source weights	0
Zero-shot NAS+FT	Up to 12 feasible C1 candidates ranked by the support proxy	Adam/Huber, 50 per candidate
H-Meta-NAS-based	240 source outer iterations; two populations of at most six feasible candidates	SGD/MSE, 50 per evaluation

Metrics and uncertainty. Mean squared error (MSE) and mean absolute error (MAE) average prediction errors over valid forecast windows and horizon steps. Within each task, Worst-10% averages the largest $\max\{1, \text{round}(0.1n)\}$ window MSEs, where n is the number of valid windows. If $0.1n$ is exactly halfway between two integers, it is rounded to the even integer. These metrics are averaged over the four tasks of each VM and then equally across VMs. Task-level CVaR90 averages the largest 12 MSEs among the 120 evaluation tasks. Thus, CVaR90 describes the tail across tasks, whereas Worst-10% describes the tail within a task.

Let $E_{\text{MSA},i}$ and $E_{\text{PT-FT},i}$ denote the MSEs of MSA-DTI and PT-FT on task i, with overbars denoting their reported averages. The aggregate MSE reduction is $100(1 - \overline{E}_{\text{MSA}}/\overline{E}_{\text{PT-FT}})$. The mean task-relative reduction instead averages $100(1 - E_{\text{MSA},i}/E_{\text{PT-FT},i})$ over paired tasks. Descriptive 95% percentile intervals for the reported MSE improvements use 10000 paired VM bootstrap draws (PCG64 seed 20261026), keeping all four tasks of each sampled VM together. Beneficial and harmful selections compare a selected alternative with the adapted reference on future-window MSE; retaining the reference contributes zero to both counts. The denominator is all evaluation tasks.

5.2. Overall Performance and Selection Reliability

Figure 5 compares all eight methods at the medium budget. MSA-DTI reduced aggregate MSE from 0.004976 for PT-FT to 0.004675, a reduction of 6.04% (95% paired VM bootstrap interval, 2.00–22.38%). MAE fell by 20.25%. Worst-10% error was slightly higher for MSA-DTI (0.029956 versus 0.029891). Task-level CVaR90 was similar for the two methods (0.038681 versus 0.038686).

Relative to PT-FT, MSA-DTI had lower MSE on 109 of 120 tasks and on 28 of 30 VMs after averaging within each VM. The mean of the individual task reductions was 54.83%. The 6.04% reduction compares the two methods' mean MSEs. Figure 6 also identifies the outcome of each selection relative to the adapted A57 reference. Of the 120 tasks, 83 improved after switching, two worsened, and 35 retained A57.

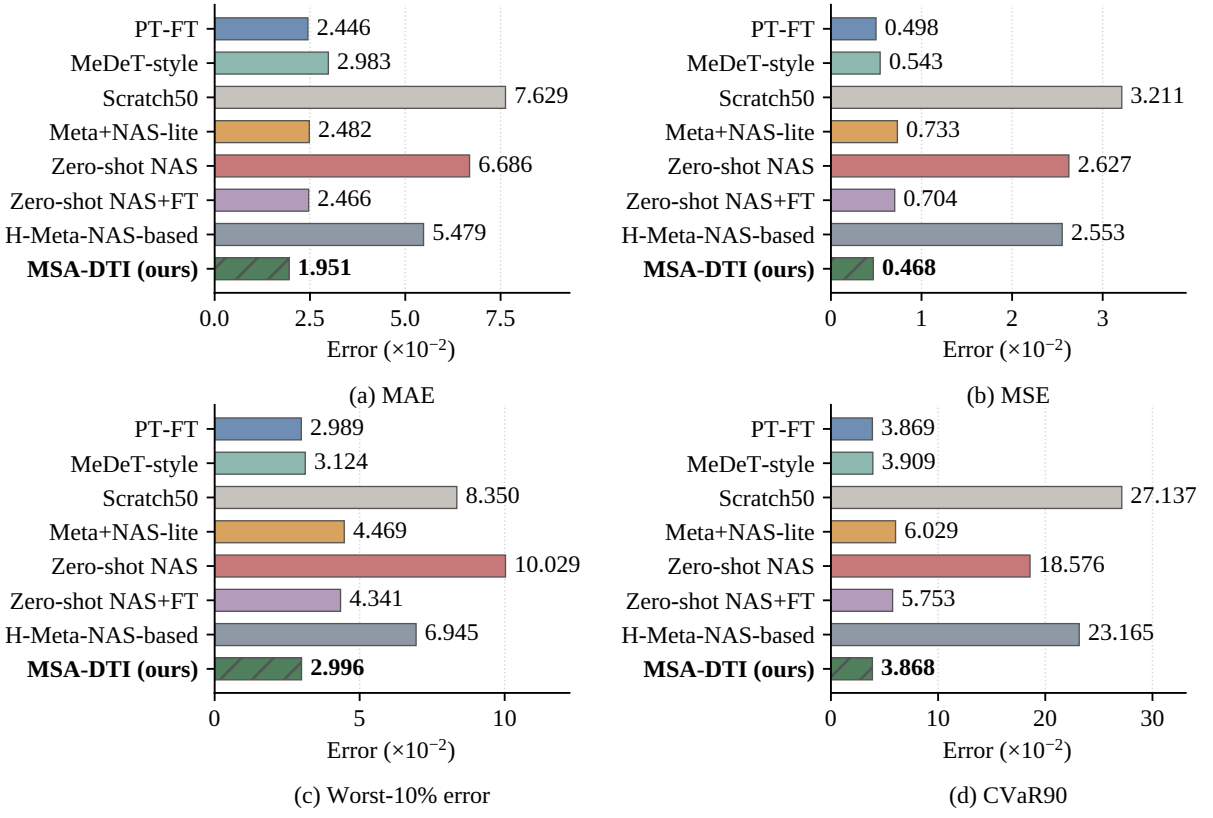


Figure 5: Prediction errors at the medium deployment budget for 120 tasks from 30 VMs. Bars show the point estimates defined in Section 5.1; lower is better.

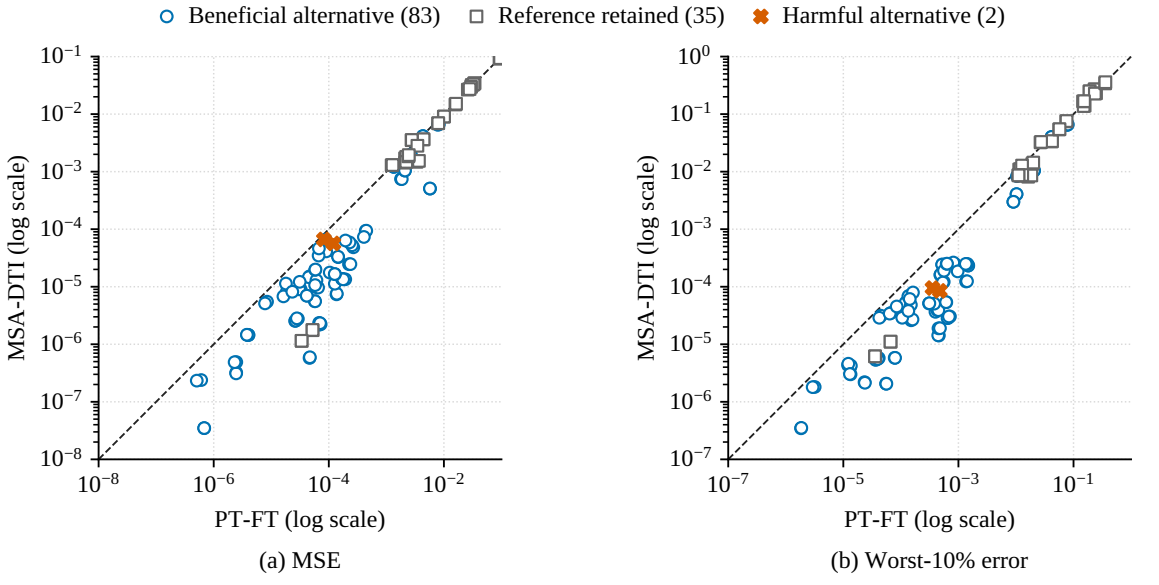
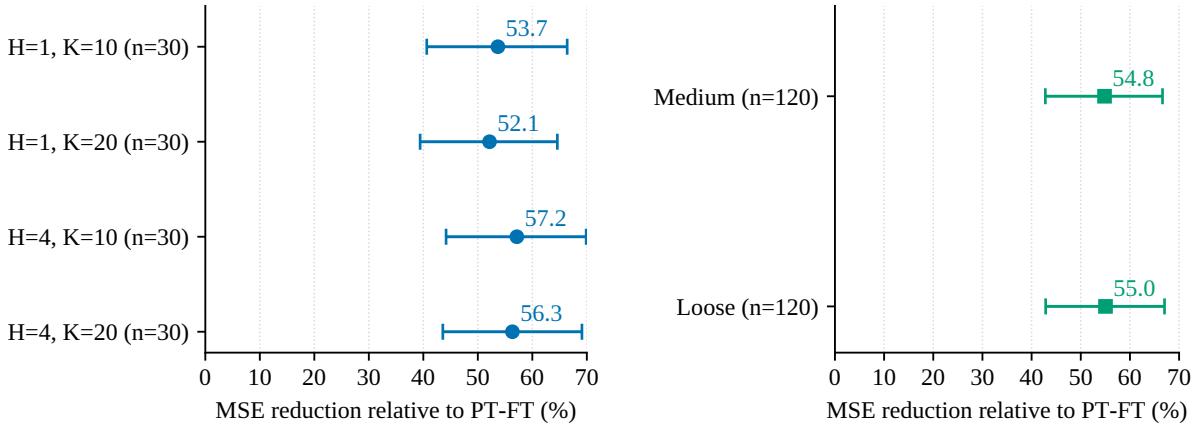


Figure 6: Paired errors for 120 evaluation tasks: (a) MSE and (b) Worst-10% error. The diagonal denotes equal errors for MSA-DTI and PT-FT. Marker categories compare MSE with the adapted A57 reference.

5.3. Robustness and Sensitivity Analysis

Figure 7 shows mean task-relative MSE reductions against fixed-backbone PT-FT. At the medium budget they range from 52.1% to 57.2% across the four horizon and support settings. The medium and loose budgets give similar reductions. Fixed A39 exceeds the tight budget, so that budget has no deployable PT-FT error for comparison.



(a) Horizon and support size (b) Deployment budget

Figure 7: Mean task-relative MSE reduction against PT-FT. Error bars show 95% paired VM-bootstrap intervals.

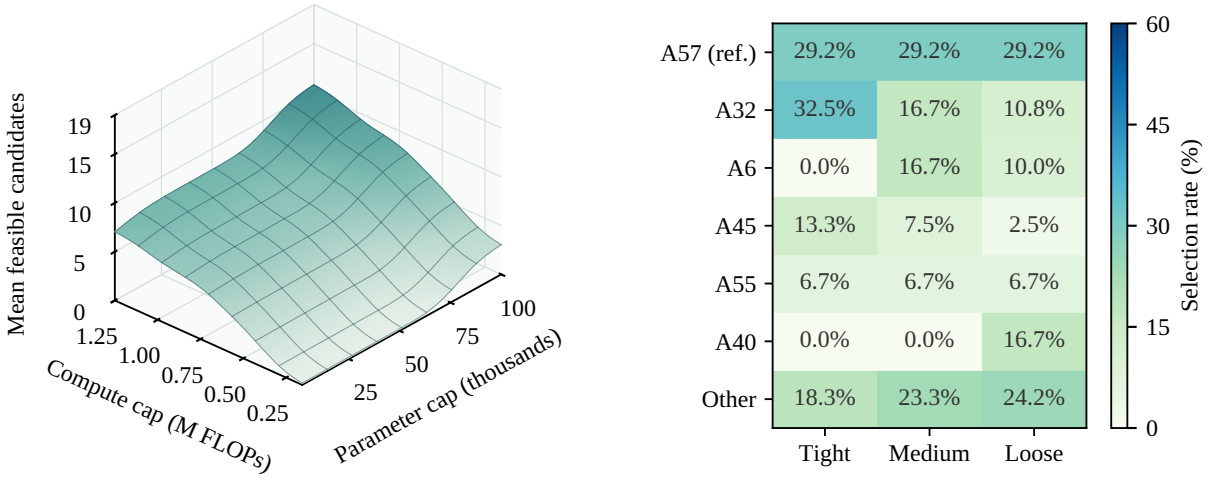
Table 3 reports the effect of the three budgets on the same 120 tasks. The tight, medium, and loose budgets admitted 10, 17, and 19 candidates. With the same calibrated rule, MSA-DTI selected eight, 12, and 14 distinct models, respectively. Aggregate MSE ranged from 0.004675 to 0.004684.

Table 3: Candidate availability and prediction errors under the three deployment budgets.

Budget	Feasible	Used	MSE	MAE	Worst-10%	CVaR90
Tight	10	8	0.004684	0.019751	0.030005	0.038681
Medium	17	12	0.004675	0.019509	0.029956	0.038681
Loose	19	14	0.004679	0.019457	0.029997	0.038681

5.4. Deployment Feasibility and Cost

Figure 8(a) shows the weighted mean number of feasible candidates at $H = 1$. The sweep covers 2,081–100,000 parameters and 155,712–1,250,000 operations. At each pair of limits, the Gaussian weighting emphasizes nearby budget pairs. The standard deviation is 10% of each axis range. Panel (b) gives the actual selection frequencies at the three experimental budgets.



(a) Weighted feasible count (H = 1)

(b) Final model selection

Figure 8: Weighted mean number of feasible candidates (a) and final model selection frequencies (b).

Figure 9 compares the selected models with both deployment limits. MSA-DTI met both limits on all 120 tasks at each budget. Selecting from all 19 candidates before checking the limits would have left 50 outputs over the tight budget and 24 over the medium budget. Fixed A39 exceeded the tight operation limit on every task but fit the other budgets. The four search baselines also met the limits across all three budgets (Table 4).

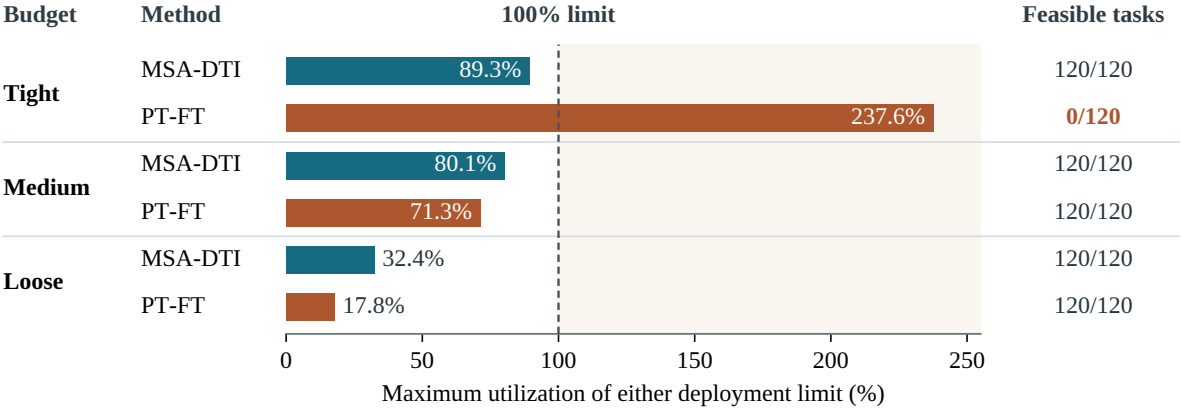


Figure 9: Maximum utilization of either deployment limit and the number of feasible tasks.

Timing in Table 4 starts at each method's first training or development stage and ends at final model selection. Shared stages count once. We excluded two documented interruptions for MSA-DTI and omitted forecast scoring for every method. MSA-DTI took 148.07 minutes and fixed PT-FT took 98.61 minutes. Host loads differed between runs.

Table 4: Deployment coverage and recorded time through model selection.

	Feasible tasks (out of 120)			Time (min)		
Method	Tight	Medium	Loose	Source/devel.	Target	Total
PT-FT	0	120	120	94.29	4.31	98.61
MeDeT-style	120	120	120	280.20	1.86	282.05
Scratch50	120	120	120	49.54	3.97	53.51
Meta+NAS-lite	120	120	120	16.59	29.31	45.89
Zero-shot NAS	120	120	120	3.81	5.74	9.55
Zero-shot NAS+FT	120	120	120	3.81	23.87	27.68
H-Meta-NAS-based	120	120	120	12.34	26.56	38.89
MSA-DTI	120	120	120	114.37	33.70	148.07

5.5. Ablation Study

The four rules in Table 5 use the same source weights for shared architectures. The error metrics include all 120 tasks, even when the selected model exceeds the budget. Without preselection, we applied the medium budget to all 66 source-trained architectures. Without deployment filtering, all 19 retained candidates enter adaptation. Without the margin, the model with the lowest validation MSE wins among the 17 feasible candidates. We measured harm against the adapted A57.

Table 5: Effect of removing one component at the medium Bitbrains budget.

			Prediction error ($\times 10^{-2}$)				
Rule	Adapted	Feasible	MSE	MAE	Worst-10%	CVaR90	Harm (%)
MSA-DTI	17	120/120	0.4675	1.9509	2.9956	3.8681	1.67
No preselection	48	120/120	0.4658	1.9486	2.9776	3.8681	1.67
No deployment filter	19	96/120	0.4679	1.9457	2.9997	3.8681	3.33
No margin	17	120/120	0.4519	1.9305	2.8329	3.6945	10.83

Preselection reduced the number of candidates requiring adaptation from 48 to 17 per task. Mean test MSE was 0.004675 with preselection and 0.004658 without it. Removing the margin lowered mean MSE to 0.004519 but increased harmful replacements from two to 13. The margin retained A57 in 31 cases where direct selection would have switched. Of those switches, 11 would have raised test MSE and 20 would have lowered it.

5.6. External Evaluation

We transferred the 19 Bitbrains candidates, the A57 reference, and the 20% margin to Microsoft Azure Public Dataset V2 (Cortez et al., 2017). All methods used the same 30 VMs, one per subscription, with the four horizon and support settings from Bitbrains. Only five-minute mean CPU utilization was observed. The other nine raw metrics were marked as missing. These subscriptions had been examined in earlier experiments. We fixed the transferred policy before scoring this run.

For Table 6, we first averaged task errors within each subscription and then took the unweighted mean across subscriptions. Fixed PT-FT exceeded the tight operation limit on every task. MSA-DTI met both deployment limits on all 120 tasks at each budget. Among feasible models, zero-shot NAS with fine-tuning had the lowest MSE and MAE at the tight and medium budgets. MSA-DTI had the lowest MSE and MAE at the loose budget.

Table 6: Prediction errors on the same 30 Azure subscriptions under three deployment budgets.

Errors ($\times 10^{-3}$)	Tight		Medium		Loose	
Method	MSE	MAE	MSE	MAE	MSE	MAE
PT-FT	Over budget		2.438	22.532	2.438	22.532
MeDeT-style	2.940	28.834	2.940	28.834	2.940	28.834
Scratch50	8.654	54.914	8.654	54.914	8.654	54.914
Meta+NAS-lite	1.608	13.107	2.312	15.411	2.673	16.283
Zero-shot NAS	30.101	139.266	8.025	59.049	5.480	45.672
Zero-shot NAS+FT	1.606	13.070	1.820	14.369	5.862	20.926
H-Meta-NAS-based	3.558	30.018	3.390	27.992	2.992	22.215
MSA-DTI	1.922	18.185	1.894	15.515	1.892	15.614

Figure 10 compares each task under adjacent budgets. At the medium budget, 79 of the 120 tasks selected an architecture excluded by the tight budget. Test MSE fell for 75 of these tasks. The median reduction across all 79 was 38.3%. At the loose budget, 11 tasks selected an architecture excluded by the medium budget. Ten had lower test MSE, with a median reduction of 4.5% across all 11.

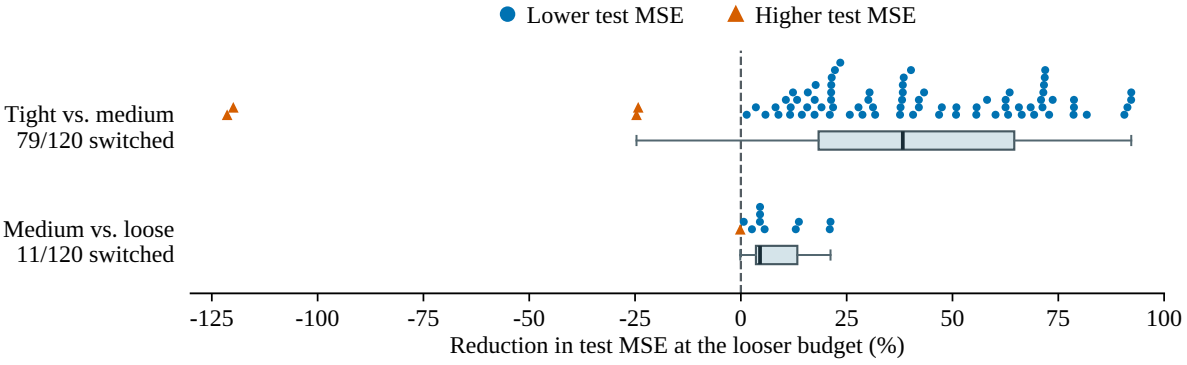


Figure 10: Task-wise changes in test MSE when MSA-DTI selects a different architecture at the looser budget.

6. Conclusion

This paper presented MSA-DTI for selecting and adapting reusable DT models under limits on parameter and operation counts. On Bitbrains, MSA-DTI met both limits on every task at all three budgets. Fixed PT-FT exceeded the operation limit at the tight budget. Under the medium budget, MSA-DTI reduced aggregate MSE by 6.04% relative to PT-FT. Its Worst-10% error was slightly higher. Compared with choosing the model with the lowest validation MSE, the calibrated rule prevented some harmful switches but also missed some beneficial ones. On Azure, most tasks that switched to a newly feasible architecture had lower test MSE. Together, feasibility filtering and comparison against an adapted reference allow a platform to select a deployable DT model for each target using a small labeled support set.

Funding

The Major Science and Technology Projects in Qinghai Province supported this work (grant 2024-GX-A3). The funder had no role in study design, data collection, analysis, or interpretation. It also had no role in preparing the manuscript or deciding to submit it.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The Bitbrains fastStorage trace is available from the Grid Workloads Archive (Bitbrains IT Services Inc., n.d.). Microsoft provides the Azure V2 trace and its schema (Microsoft, 2019). The MSA-DTI release (review-v1) (MSA-DTI, 2026) provides the experimental code, task splits, model weights, per-task predictions, and scripts to reproduce the figures and tables.

AI Assistance with Language Editing

The authors used OpenAI ChatGPT and Codex to improve the clarity of the writing.

Acknowledgements

The authors acknowledge Bitbrains IT Services Inc. for providing the workload traces and the Grid Workloads Archive for making them available (Shen et al., 2015).

References

[dataset] Bitbrains IT Services Inc., n.d. GWA-T-12 BitBrains, fastStorage trace. Grid Workloads Archive. https://atlarge-research.com/gwa-t-12/ (accessed 5 November 2026).

Bornschein, J., Visin, F., Osindero, S., 2020. Small data, big decisions: Model selection in the small-data regime. In: *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, vol. 119, pp. 1035–1044. https://proceedings.mlr.press/v119/bornschein20a.html.

Cortez, E., Bonde, A., Muzio, A., Russinovich, M., Fontoura, M., Bianchini, R., 2017. Resource Central: Understanding and predicting workloads for improved resource management in large cloud platforms. In: *Proc. 26th Symp. Oper. Syst. Princ. (SOSP)*, pp. 153–167. https://doi.org/10.1145/3132747.3132772.

Emami Khansari, M., Sharifian, S., 2025. A deep reinforcement learning approach towards distributed Function as a Service (FaaS) based edge application orchestration in cloud-edge continuum. *J. Netw. Comput. Appl.* 233, 104042. https://doi.org/10.1016/j.jnca.2024.104042.

Eustratiadis, P., Dudziak, Ł., Li, D., Hospedales, T., 2024. Neural fine-tuning search for few-shot learning. In: *Proc. Int. Conf. Learn. Represent. (ICLR)*. https://openreview.net/forum?id=T7YV5UZKBc.

Gharoun, H., Momenifar, F., Chen, F., Gandomi, A.H., 2024. Meta-learning approaches for few-shot learning: A survey of recent advances. *ACM Comput. Surv.* 56 (12), 294, 1–41. https://doi.org/10.1145/3659943.

Ghavamzadeh, M., Petrik, M., Chow, Y., 2016. Safe policy improvement by minimizing robust baseline regret. In: *Adv. Neural Inf. Process. Syst.*, vol. 29, pp. 2298–2306. https://proceedings.neurips.cc/paper_files/paper/2016/hash/9a3d458322d70046f63dfd8b0153ece4-Abstract.html.

Giannetti, C., Essien, A., 2022. Towards scalable and reusable predictive models for cyber twins in manufacturing systems. *J. Intell. Manuf.* 33 (2), 441–455. https://doi.org/10.1007/s10845-021-01804-0.

Hakiri, A., Gokhale, A., Ben Yahia, S., Mellouli, N., 2024. A comprehensive survey on digital twin for future networks and emerging Internet of Things industry. *Comput. Netw.* 244, 110350. https://doi.org/10.1016/j.comnet.2024.110350.

Hortelano, D., de Miguel, I., Durán Barroso, R.J., Aguado, J.C., Merayo, N., Ruiz, L., Asensio, A., Masip-Bruin, X., Fernández, P., Lorenzo, R.M., Abril, E.J., 2023. A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems. *J. Netw. Comput. Appl.* 216, 103669. https://doi.org/10.1016/j.jnca.2023.103669.

Hunter, J.D., 2007. Matplotlib: A 2D graphics environment. *Comput. Sci. Eng.* 9 (3), 90–95. https://doi.org/10.1109/MCSE.2007.55.

Khan, L.U., Han, Z., Saad, W., Hossain, E., Guizani, M., Hong, C.S., 2022. Digital twin of wireless systems: Overview, taxonomy, challenges, and opportunities. *IEEE Commun. Surv. Tutorials* 24 (4), 2230–2254. https://doi.org/10.1109/COMST.2022.3198273.

Lacoste, A., Marchand, M., Laviolette, F., Larochelle, H., 2014. Agnostic Bayesian learning of ensembles. In: *Proc. 31st Int. Conf. Mach. Learn. (ICML)*, vol. 32 (1), pp. 611–619. https://proceedings.mlr.press/v32/lacoste14.html.

Li, G., Hoang, D., Bhardwaj, K., Lin, M., Wang, Z., Marculescu, R., 2024. Zero-shot neural architecture search: Challenges, solutions, and opportunities. *IEEE Trans. Pattern Anal. Mach. Intell.* 46 (12), 7618–7635. https://doi.org/10.1109/TPAMI.2024.3395423.

Li, J., Guo, S., Liang, W., Wang, J., Chen, Q., Zeng, Y., Ye, B., Jia, X., 2024. Digital twin-enabled service provisioning in edge computing via continual learning. *IEEE Trans. Mob. Comput.* 23 (6), 7335–7350. https://doi.org/10.1109/TMC.2023.3332668.

Lian, D., Zheng, Y., Xu, Y., Lu, Y., Lin, L., Zhao, P., Huang, J., Gao, S., 2020. Towards fast adaptation of neural architectures with meta learning. In: *Proc. Int. Conf. Learn. Represent. (ICLR)*. https://openreview.net/forum?id=r1eowANFvr.

The Matplotlib Development Team, 2024. Matplotlib: Visualization with Python (version 3.10.0) [software]. Zenodo. https://doi.org/10.5281/zenodo.14464227.

[dataset] Microsoft, 2019. Azure Public Dataset V2 [version 2]. Microsoft Azure Traces. https://github.com/Azure/AzurePublicDataset/blob/master/AzurePublicDatasetV2.md (accessed 5 November 2026).

Mihai, S., Yaqoob, M., Hung, D.V., Davis, W., Towakel, P., Raza, M., Karamanoglu, M., Barn, B., Shetve, D., Prasad, R.V., Venkataraman, H., Trestian, R., Nguyen, H.X., 2022. Digital twins: A survey on enabling technologies, challenges, trends and future prospects. *IEEE Commun. Surv. Tutorials* 24 (4), 2255–2291. https://doi.org/10.1109/COMST.2022.3208773.

[dataset] MSA-DTI, 2026. MSA-DTI manuscript and reproduction materials (review-v1). GitHub. https://github.com/w924871681/MSA-DTI/releases/tag/review-v1 (accessed 5 November 2026).

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. PyTorch: An imperative style, high-performance deep learning library. In: *Adv. Neural Inf. Process. Syst.*, vol. 32, pp. 8024–8035. https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

PyTorch contributors, 2024. PyTorch (version 2.5.1) [software]. GitHub. https://github.com/pytorch/pytorch/releases/tag/v2.5.1 (accessed 5 November 2026).

Qin, B., Pan, H., Dai, Y., Si, X., Huang, X., Yuen, C., Zhang, Y., 2024. Machine and deep learning for digital twin networks: A survey. *IEEE Internet Things J.* 11 (21), 34694–34716. https://doi.org/10.1109/JIOT.2024.3416733.

Sartaj, H., Ali, S., Gjøby, J.M., 2025. MeDeT: Medical device digital twins creation with few-shot meta-learning. *ACM Trans. Softw. Eng. Methodol.* 34 (6), 158, 1–36. https://doi.org/10.1145/3708534.

Shen, S., van Beek, V., Iosup, A., 2015. Statistical characterization of business-critical workloads hosted in cloud datacenters. In: *Proc. 15th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGrid)*, pp. 465–474. https://doi.org/10.1109/CCGrid.2015.60.

Son, J., Lee, S., Kim, G., 2025. When meta-learning meets online and continual learning: A survey. *IEEE Trans. Pattern Anal. Mach. Intell.* 47 (1), 413–432. https://doi.org/10.1109/TPAMI.2024.3463709.

Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A., Le, Q.V., 2019. MnasNet: Platform-aware neural architecture search for mobile. In: *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 2815–2823. https://doi.org/10.1109/CVPR.2019.00293.

Tao, F., Zhang, H., Liu, A., Nee, A.Y.C., 2019. Digital twin in industry: State-of-the-art. *IEEE Trans. Ind. Informatics* 15 (4), 2405–2415. https://doi.org/10.1109/TII.2018.2873186.

Vettoruzzo, A., Bouguelia, M.-R., Vanschoren, J., Rögnvaldsson, T., Santosh, K.C., 2024. Advances and challenges in meta-learning: A technical review. *IEEE Trans. Pattern Anal. Mach. Intell.* 46 (7), 4763–4779. https://doi.org/10.1109/TPAMI.2024.3357847.

Wang, W., Duan, L., Wang, Y., Fan, J., Zhang, Z., 2023. MMT: Cross domain few-shot learning via meta-memory transfer. *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (12), 15018–15035. https://doi.org/10.1109/TPAMI.2023.3306352.

Wu, B., Dai, X., Zhang, P., Wang, Y., Sun, F., Wu, Y., Tian, Y., Vajda, P., Jia, Y., Keutzer, K., 2019. FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search. In: *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 10726–10734. https://doi.org/10.1109/CVPR.2019.01099.

Wu, J., Liu, X., Yin, X., Zhang, T., Zhang, Y., 2024. Task-adaptive prompted transformer for cross-domain few-shot learning. In: *Proc. AAAI Conf. Artif. Intell.*, vol. 38 (6), pp. 6012–6020. https://doi.org/10.1609/aaai.v38i6.28416.

Wu, M.-T., Lin, H.-I., Tsai, C.-W., 2024. A training-free neural architecture search algorithm based on search economics. *IEEE Trans. Evol. Comput.* 28 (2), 445–459. https://doi.org/10.1109/TEVC.2023.3264533.

Wu, Y., Zhang, K., Zhang, Y., 2021. Digital twin networks: A survey. *IEEE Internet Things J.* 8 (18), 13789–13804. https://doi.org/10.1109/JIOT.2021.3079510.

Xu, H., Wu, J., Pan, Q., Guan, X., Guizani, M., 2023. A survey on digital twin for Industrial Internet of Things: Applications, technologies and tools. *IEEE Commun. Surv. Tutorials* 25 (4), 2569–2598. https://doi.org/10.1109/COMST.2023.3297395.

You, K., Liu, Y., Wang, J., Long, M., 2021. LogME: Practical assessment of pre-trained models for transfer learning. In: *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139, pp. 12133–12143. https://proceedings.mlr.press/v139/you21b.html.

You, K., Liu, Y., Zhang, Z., Wang, J., Jordan, M.I., Long, M., 2022. Ranking and tuning pre-trained models: A new paradigm for exploiting model hubs. *J. Mach. Learn. Res.* 23 (209), 1–47. https://www.jmlr.org/papers/v23/21-1251.html.

Zhao, Y., Wang, L., Tian, Y., Fonseca, R., Guo, T., 2021. Few-shot neural architecture search. In: *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139, pp. 12707–12718. https://proceedings.mlr.press/v139/zhao21d.html.

Zhao, Y., Gao, X., Shumailov, I., Fusi, N., Mullins, R., 2022. Rapid model architecture adaption for meta-learning. In: *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 18721–18732. https://doi.org/10.52202/068431-1360.

Zhao, Y., Zhang, T., Li, J., Tian, Y., 2023. Dual adaptive representation alignment for cross-domain few-shot learning. *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (10), 11720–11732. https://doi.org/10.1109/TPAMI.2023.3272697.

Zhao, Y., Wang, L., Guo, T., 2024. Multi-objective neural architecture search by learning search space partitions. *J. Mach. Learn. Res.* 25 (177), 1–41. https://www.jmlr.org/papers/v25/23-1013.html.

Zintgraf, L.M., Shiarlis, K., Kurin, V., Hofmann, K., Whiteson, S., 2019. Fast context adaptation via meta-learning. In: *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, vol. 97, pp. 7693–7702. https://proceedings.mlr.press/v97/zintgraf19a.html.

Zuo, Z., Huang, Y., Li, Z., Jiang, Y., Liu, C., 2025. Mixed contrastive transfer learning for few-shot workload prediction in the cloud. *Computing* 107 (1), 5. https://doi.org/10.1007/s00607-024-01366-y.